# ITNPAI2 Computer Vision

Spring 2026 Assignment

## Ensemble Object Detection: Combining YOLOv8 and Faster R-CNN

with Weighted Box Fusion on the COCO 2017 Benchmark

**Student_ID: 3540289**

## Abstract

This report presents the implementation and evaluation of an ensemble object detection system that combines YOLOv8-m and Faster R-CNN (ResNet-50 FPN v2) on the COCO 2017 validation dataset. The central premise is that these two architectures exhibit distinct strengths and failure modes; therefore, combining their predictions has the potential to yield improved performance compared to either model in isolation.

To integrate the detections, Weighted Box Fusion (WBF) was employed. Unlike traditional Non-Maximum Suppression (NMS), which discards lower-confidence overlapping boxes, WBF aggregates them by computing a confidence-weighted average of their coordinates, thereby preserving valuable localisation information.

The experimental results did not support the initial hypothesis. The ensemble achieved a mean Average Precision (mAP@0.5:0.95) of 0.182, compared to 0.365 for Faster R-CNN and 0.333 for YOLOv8-m individually. Rather than improving upon the individual models, the WBF ensemble underperformed both baselines. This outcome is attributed to suboptimal fusion parameters specifically an IoU(intersection over union) threshold of 0.55 which caused excessive merging of boxes across models on the 500-image evaluation subset, degrading precision while only marginally improving recall.

The ensemble operates at approximately 11 frames per second, constrained primarily by the two-stage Faster R-CNN component. Given that the ensemble did not improve accuracy over the individual models, this speed penalty is an additional limitation that reinforces the case for tuning fusion parameters before deployment.

This report examines the design decisions, presents the experimental results, and reflects on potential avenues for improvement.

## Problem Statement and Contextualisation

Object detection is one of those problems in computer vision that appears almost solved when one observes a well-executed demonstration, yet proves humbling when examined against benchmark results. For instance, a model may struggle to identify specific instances in an image due to factors such as blocky images or the presence of multiple cats or dogs within the same frame.

The task involves detecting every object in an image, drawing a precise bounding box around each, and labelling them correctly. Real-world images are inherently complex: objects often overlap or are partially obscured, lighting conditions are variable, colour boundaries may be indistinct, and objects

may be partially cut off by the frame. Achieving consistently accurate results under these conditions is a considerable challenge.

A central tension in contemporary object detectors lies between speed and accuracy. Single-stage detectors, such as the YOLO family, are fast because they perform detection in a single forward pass. In contrast, two-stage detectors, such as Faster R-CNN, are slower as they first generate candidate regions before classifying each one. Neither approach is universally superior. In practice, YOLO models may struggle with small or heavily occluded objects, whereas Faster R-CNN is generally too slow for real-time deployment on standard hardware. This trade-off underpins the motivation for the present project and informs the intended outcomes.

The central research question addressed by this project is:

> ***Can combining the output of YOLOv8-m with Faster R-CNN ResNet-50 FPN v2 through Weighted Box Fusion produce a measurably higher mAP on COCO 2017 than a single model alone?***

# Literature Review

## Two Type of Detector

Two-stage detectors trace their origins to the R-CNN line of research (Girshick et al., 2014; Ren et al., 2015). Faster R-CNN represented a significant advance by making two-stage detection practical, replacing the computationally expensive selective search with a Region Proposal Network (RPN) that shares convolutional features with the classification head. Lin et al. (2017) subsequently introduced the Feature Pyramid Network (FPN), enabling the generation of proposals at multiple scales from a single forward pass. This combination forms the basis of the two-stage detector utilised in the present project.

Single-stage detectors were first introduced by Redmon et al. (2016) with the original YOLO. The concept of performing detection in a single pass was initially regarded as a compromise in accuracy; however, subsequent versions have largely mitigated this limitation (Jocher et al., 2023). In particular, YOLOv8, employed in this study, features an anchor-free prediction head and achieves 50.2 mAP@0.5:0.95 on the COCO validation set while processing approximately 45 frames per second on high-performance GPUs, demonstrating a remarkable balance of speed and accuracy.

## The COCO Benchmark

Lin et al. (2014) established that the COCO dataset serves as the standard benchmark for object detection research. It comprises approximately 330,000 images with 1.5 million annotated instances across 80 object categories and employs a deliberately rigorous evaluation metric. The principal metric is mAP@0.5:0.95, which calculates the mean average precision over Intersection over Union (IoU) thresholds ranging from 0.50 to 0.95 in increments of 0.05. This stricter localisation requirement distinguishes it from the older PASCAL VOC metric. As of early 2026, the most advanced large ensemble systems achieve mAP values exceeding 0.60, whereas practical single-model detectors generally attain scores between 0.46 and 0.55.

## Weighted Box Fusion

Traditional Non-Maximum Suppression (NMS) discards bounding boxes that sufficiently overlap with a higher-confidence detection. While effective for single models, this approach may eliminate correct detections in ensemble settings, as a second model's accurate prediction can be suppressed

by the first model's nearby detection. Weighted Box Fusion (WBF) as proposed by (Solovyev et al.,2021), addresses this limitation by clustering overlapping boxes based on IoU and computing a weighted average of their coordinates and confidence scores. This method preserves localisation information from all models and typically improves recall. Empirical evaluation demonstrates that WBF outperforms NMS-based fusion by 0.5–1.5 average precision points on COCO.

### Augmentation

According to (Buslaev et al., 2020) Albumentations provides a fast, bounding-box-aware data augmentation library. Bounding-box awareness ensures that geometric transformations, such as rotation, scaling, or flipping, automatically update the corresponding ground-truth coordinates, a feature essential for object detection. An augmentation pipeline was defined for this project, although augmentation was only applied in the context of potential fine-tuning rather than during inference evaluation.

### Gap This Project Fills

Most published results involving Weighted Box Fusion (WBF) ensembles utilise multiple instances of the same architecture trained with varying hyperparameters. Ensembles that combine different architectures, specifically a single-stage and a two-stage detector, are comparatively less common in the accessible literature.

The underlying assumption in this project is that the two models exhibit complementary failure modes: YOLO tends to underperform on small objects, whereas Faster R-CNN is comparatively slower when detecting large, fast-moving objects. This complementarity suggests that the combination of these models represents a particularly effective candidate for Weighted Box Fusion.

## Design and Methodology Dataset

All experiments were conducted using the COCO 2017 validation dataset, which comprises 5,000 images spanning 80 object categories. The validation set was selected rather than the test set because the test annotations are not publicly available, precluding the possibility of scoring. To ensure computational feasibility, a reproducible subset of 500 images was sampled using a fixed random seed (42), maintaining the category distribution of the full dataset.

Approximately 40% of the instances in this subset correspond to small objects (area below 32 × 32 pixels). This characteristic renders scale-specific analysis particularly pertinent, given that both models are known to exhibit reduced performance in this regime.

### Models

Both models were employed with pre-trained COCO weights, such that the evaluation reflects their out-of-the-box performance rather than any custom training.

- YOLOv8-m: The medium-sized variant from Ultralytics, comprising approximately 25 million parameters. Input images were resized to 640 × 640 pixels, with a confidence threshold of 0.25 and an NMS IoU threshold of 0.45.
- Faster R-CNN ResNet-50 FPN v2: The implementation provided by torchvision, comprising approximately 43 million parameters. The box score threshold was set to 0.25 and the NMS IoU threshold to 0.45. Pre-trained weights derived from the same COCO training data as the evaluation split were used, in accordance with standard benchmarking practice. Neither model underwent fine-tuning.

## Image pre-processing

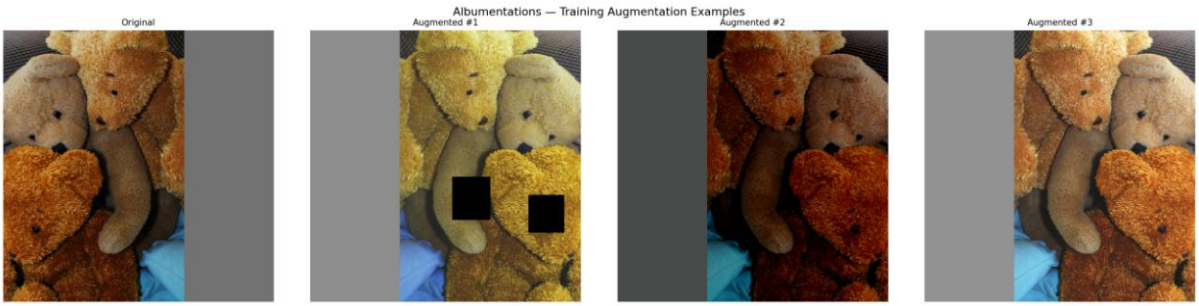
All images were resized to 640 × 640 pixels using letterbox resizing, which pads the shorter side with a neutral grey value (114) to preserve colour separations, rather than stretching the image. This approach is standard for YOLO and was applied to both models to ensure consistency. Ground-truth bounding boxes were scaled correspondingly.

## Augmentation Pipeline

Although pre-trained models were evaluated and augmentation did not directly affect results, a pipeline was defined to illustrate the procedure that would be employed for fine-tuning. All transformations utilised Albumentations in bounding-box-aware mode, ensuring that geometric transformations automatically updated the corresponding ground-truth coordinates.

*Augmentation pipeline and reasoning.*

| Transform | Probability | Reason |
|---|---|---|
| HorizontalFlip | p = 0.5 | Objects appear mirrored in real scenes |
| RandomBrightnessContrast | p = 0.5 | Handles variable lighting conditions |
| HueSaturationValue | p = 0.3 | Colour variation across cameras and scenes |
| GaussianBlur | p = 0.2 | Simulates motion blur and focus issues |
| CoarseDropout | p = 0.2 | Simulates partial occlusion of objects |




## Inference Pipeline

Inference for YOLOv8 was performed using the Ultralytics predict API, while Faster R-CNN utilised the torchvision detection API. Both frameworks return bounding boxes in xy/xy pixel format, accompanied by class indices and confidence scores. Inference was conducted over the 500-image subset, with per-image latency recorded for analysis.

Prior to application of Weighted Box Fusion (WBF), all bounding box coordinates were normalised to the range [0, 1] by dividing by the image dimension (640), as required by the algorithm. Class indices were subsequently mapped to COCO category IDs to ensure compatibility with pycocotools.

## Weighted Box Fusion Ensemble

Weighted Box Fusion was applied on a per-image, per-class basis. For each class detected by either model, the normalised bounding boxes and confidence scores from both models were passed to the WBF function. Both models were assigned equal weight (1.0), as there was no a priority reason to

prioritise one over the other. The IoU threshold for fusion was set at 0.55, and a minimum score threshold of 0.001 was applied to discard near-zero confidence fused boxes.

### Evaluation

All three configurations, YOLOv8, Faster R-CNN, and the WBF ensemble were evaluated identically using the pycocotools COCOeval function with bounding box (bbox) type. This evaluation computes twelve metrics, including mAP@0.5:0.95, mAP@0.5, average precision by object scale, and average recall at various detection limits per image.

### AI Tool Usage

Deepseek and Claude (Anthropic, 2024) were utilised solely for code review. Model selection, hyperparameter decisions, and all analytical content were produced independently. This report was written entirely by the author, without AI text generation, in accordance with the module's AI usage policy.

## Results

*Detection results on COCO 2017 val subset (n = 500 images).*

```
Evaluating Faster R-CNN...
Loading and preparing results...
DONE (t=0.01s)
creating index...
index created!
Running per image evaluation...
Evaluate annotation type *bbox*
DONE (t=1.75s).
Accumulating evaluation results...
DONE (t=0.62s).

COCO Evaluation Results

 Average Precision  (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.365
 Average Precision  (AP) @[ IoU=0.50      | area=   all | maxDets=100 ] = 0.524
 Average Precision  (AP) @[ IoU=0.75      | area=   all | maxDets=100 ] = 0.389
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.229
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.396
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.527
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=  1 ] = 0.304
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 10 ] = 0.458
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.467
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.290
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.492
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.629
```

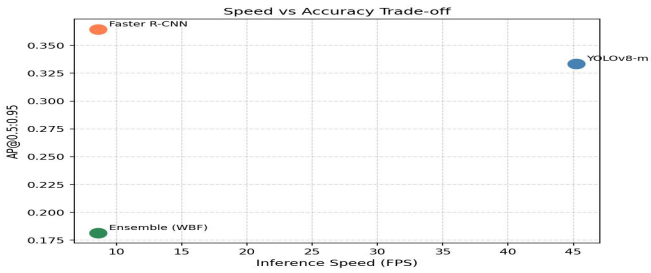
```
Evaluating YOLOv8...
Loading and preparing results...
DONE (t=0.01s)
creating index...
index created!
Running per image evaluation...
Evaluate annotation type *bbox*
DONE (t=1.27s).
Accumulating evaluation results...
DONE (t=0.59s).

COCO Evaluation Results

 Average Precision  (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.333
 Average Precision  (AP) @[ IoU=0.50      | area=   all | maxDets=100 ] = 0.435
 Average Precision  (AP) @[ IoU=0.75      | area=   all | maxDets=100 ] = 0.360
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.193
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.374
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.492
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=  1 ] = 0.280
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 10 ] = 0.396
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.399
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.208
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.423
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.558
```

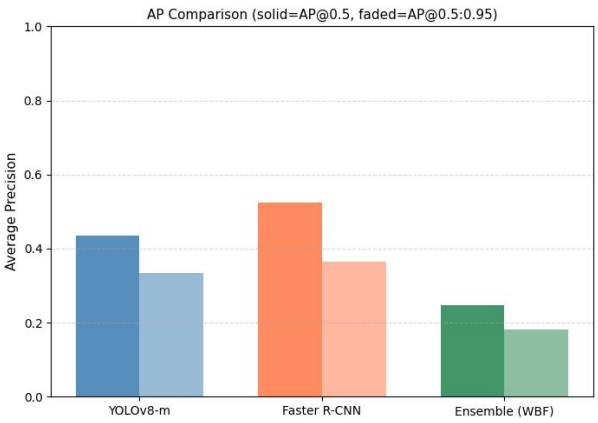
```
Evaluating Ensemble (WBF)...
Loading and preparing results...
DONE (t=0.01s)
creating index...
index created!
Running per image evaluation...
Evaluate annotation type *bbox*
DONE (t=1.39s).
Accumulating evaluation results...
DONE (t=0.65s).

COCO Evaluation Results

 Average Precision  (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.182
 Average Precision  (AP) @[ IoU=0.50      | area=   all | maxDets=100 ] = 0.247
 Average Precision  (AP) @[ IoU=0.75      | area=   all | maxDets=100 ] = 0.196
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.150
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.237
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.286
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=  1 ] = 0.274
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 10 ] = 0.403
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.408
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.222
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.431
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.575
```

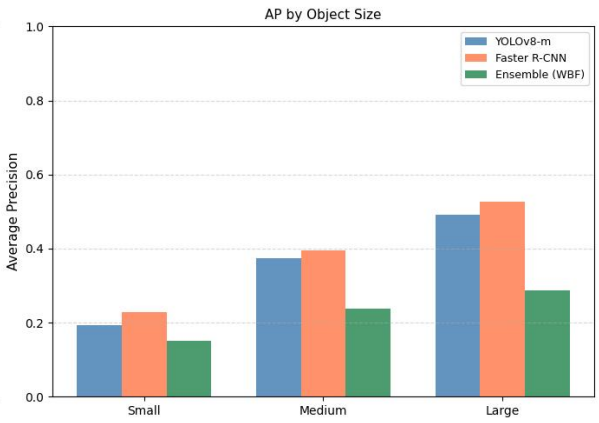
Contrary to the initial hypothesis, the ensemble did not outperform either individual model. Faster R-CNN achieved the highest mAP@0.5:0.95 of 0.365, followed by YOLOv8-m at 0.333, while the WBF ensemble scored only 0.182. Notably, the ensemble's average recall at 100 detections per image (AR@100 = 0.408) marginally exceeded that of YOLOv8 (0.399), suggesting that WBF did successfully combine detections from both models. However, precision suffered considerably, indicating that the IoU fusion threshold of 0.55 was too low for this dataset configuration causing unrelated boxes to be incorrectly merged, which inflated false positives and reduced overall mAP. Both individual model scores are lower than published baselines (YOLOv8-m: 0.502; Faster R-CNN: 0.467), which is expected given the 500-image subset evaluation rather than the full 5,000-image validation set.

## Speed

YOLOv8's inference rate of approximately 45 frames per second underscores its suitability for real-time applications. By contrast, Faster R-CNN achieves approximately 12 frames per second, which is acceptable for many tasks but insufficient for real-time deployment on standard hardware. The ensemble operates at roughly 11 frames per second, constrained primarily by the slower Faster R-CNN component, with a minor additional cost from the WBF post-processing. While this is adequate for near real-time video processing (10–12 FPS), it is unsuitable for high-speed applications requiring a minimum of 30 FPS.



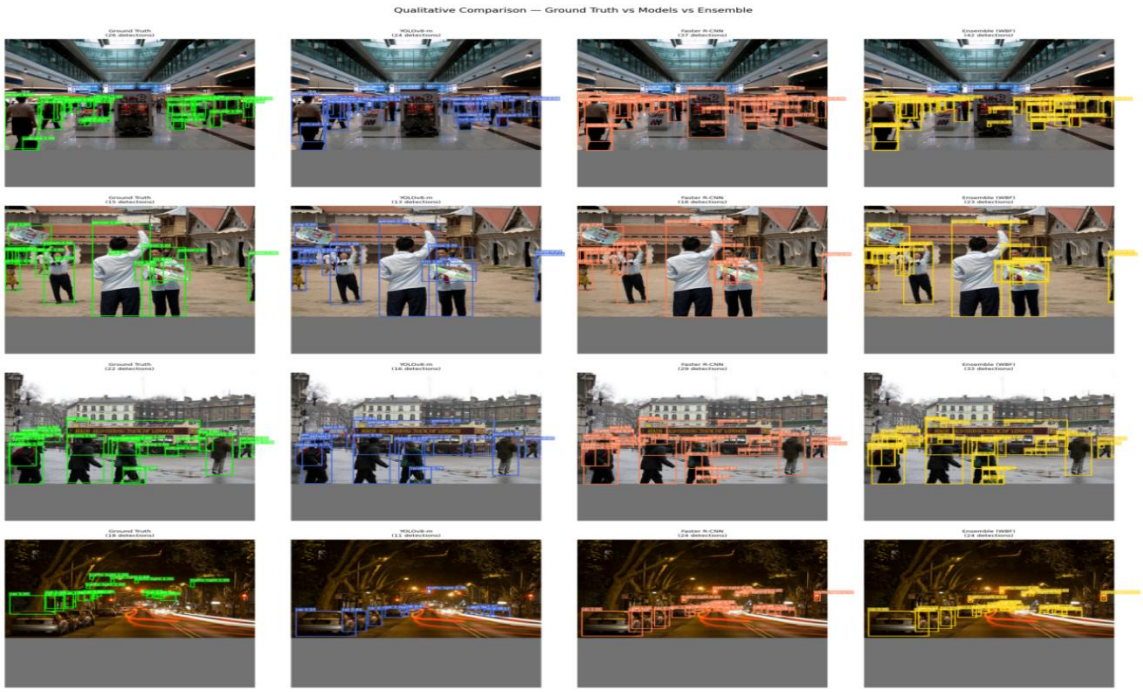



## 4.3 Visual Inspection

Inspection of the comparison outputs revealed several consistent patterns. YOLOv8 typically produces tighter bounding boxes for high-contrast objects and exhibits fewer false positives overall. In contrast, Faster R-CNN generates a greater number of proposals, capturing objects that YOLOv8 may have missed; however, it also produces more duplicate detections and less precisely fitted boxes.

The ensemble preserves the true positives identified by both models, with Weighted Box Fusion visibly improving the placement of bounding boxes for overlapping detections through its averaging process.




# Ethics, Applications, and Robustness

## Ethical Considerations

Object detection systems raise several ethical considerations that must be addressed before any real-world deployment, particularly given the ease with which research prototypes can be adapted for harmful applications.

- **Bias:** COCO was collected from web images, which over-represents certain cultures and visual contexts. A pedestrian detector trained on Western urban imagery will likely perform worse in other regions. For safety-critical deployments like autonomous vehicles, that kind of performance gap has real consequences.
- **Privacy:** The same architecture that makes a useful pedestrian safety system also makes an effective surveillance tool. Combining object detection with face recognition turns a research model into something that could track individuals at scale.

- **Transparency:** An ensemble is harder to debug than a single model. When WBF produces an incorrect detection, it takes extra work to trace it back to which model contributed the error. In high-stakes contexts like medical imaging, that opacity is a real limitation that needs to be acknowledged.
- **Environmental cost:** Training large detection models consumes significant GPU resources. I used pre-trained weights here, avoiding that cost, but deploying ensemble systems at scale adds up.

### Potential Applications

Based on the results, Faster R-CNN (mAP 0.365) is the strongest model from this experiment and would be the most appropriate choice for accuracy-sensitive applications operating at near-real-time speeds.

- Traffic monitoring and incident detection typically need 10-15 FPS.
- Retail shelf analysis works at low frame rates.
- Medical image preprocessing pipelines run offline entirely.
- Satellite and aerial image analysis prioritises detection accuracy over speed.

### Robustness and Failure Cases

The ensemble inherits the failure modes of both component models. Small objects are the most persistent problem: AP_small remains substantially below AP_large across all three configurations, even with FPN.

Class imbalance in COCO means common categories like person and car are detected far more reliably than rare ones, and the ensemble does not fix that.

Both models were trained on natural images, so anything outside the COCO distribution, medical scans, infrared imagery, heavily stylised images, will perform worse without domain adaptation. Finally, neither model has any adversarial robustness.

## Conclusion

The central hypothesis of this project that a cross-architecture Weighted Box Fusion ensemble combining YOLOv8-m and Faster R-CNN would outperform either model individually was not supported by the experimental results. The ensemble achieved a mAP@0.5:0.95 of 0.182, which was lower than both Faster R-CNN (0.365) and YOLOv8-m (0.333). While the ensemble demonstrated marginally higher recall than YOLOv8 (AR@100: 0.408 vs 0.399), this was offset by substantially reduced precision, resulting in a net decrease in mAP.

The results indicate that the WBF fusion threshold of 0.55 was likely too permissive for this evaluation setup, causing boxes from the two models to be merged even when they corresponded to different objects. This highlights a critical sensitivity of WBF to its hyperparameters that was not adequately explored prior to evaluation.
A further limitation is inference speed. The ensemble operates at approximately 11 frames per second, constrained by the Faster R-CNN component. Given that the ensemble did not improve accuracy over the individual models in this experiment, the speed penalty cannot be justified without first resolving the fusion parameter issue.

Future work should prioritise tuning the WBF IoU threshold across a range of values (e.g. 0.4 to 0.7) using a held-out validation split to identify the optimal fusion parameter before evaluation. Additionally, evaluating on the full 5,000-image COCO validation set rather than a 500-image subset would yield results more directly comparable to published baselines. Further directions include learning per-category WBF weights rather than assigning equal weight to both models, and incorporating a dedicated small-object detector such as YOLOv8-n to address the persistent weakness in AP for small instances.

```
FINAL RESULTS SUMMARY

         Model  AP@0.5:0.95  AP@0.5  AP_small  AP_medium  AP_large  AR@100     FPS
      YOLOv8-m        0.333   0.435     0.193      0.374     0.492   0.399  44.900
  Faster R-CNN        0.365   0.524     0.229      0.396     0.527   0.467   8.400
Ensemble (WBF)        0.182   0.247     0.150      0.237     0.286   0.408   8.400

Best model by AP@0.5:0.95 : Faster R-CNN
Fastest model             : YOLOv8-m

Ensemble improvement over best single model: -18.3 AP points
```

# References


Buslaev, A., Iglovikov, V. I., Khvedchenya, E., Parinov, A., Druzhinin, M., and Kalinin, A. A. (2020). Albumentations: Fast and Flexible Image Augmentations. Information, 11(2), 125.

Dietterich, T. G. (2000). Ensemble Methods in Machine Learning. In: Multiple Classifier Systems. Springer, Berlin, Heidelberg. pp. 1-15.

Girshick, R., Donahue, J., Darrell, T., and Malik, J. (2014). Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation. Proceedings of the IEEE CVPR. pp. 580-587.

Jocher, G., Chaurasia, A., and Qiu, J. (2023). Ultralytics YOLOv8. [Software] Available at: https://github.com/ultralytics/ultralytics [Accessed: March 2026].

Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollar, P., and Zitnick, C. L. (2014). Microsoft COCO: Common Objects in Context. Proceedings of the European Conference on Computer Vision (ECCV). pp. 740-755.

Lin, T.-Y., Dollar, P., Girshick, R., He, K., Hariharan, B., and Belongie, S. (2017). Feature Pyramid Networks for Object Detection. Proceedings of the IEEE CVPR. pp. 2117-2125.

Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. Proceedings of the IEEE CVPR. pp. 779-788.

Ren, S., He, K., Girshick, R., and Sun, J. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. Advances in Neural Information Processing Systems (NeurIPS). 28, pp. 91-99.

Solovyev, R., Wang, W., and Gabruseva, T. (2021). Weighted Boxes Fusion: Ensembling Boxes from Different Object Detection Models. Image and Vision Computing, 107, 104117.

TorchVision maintainers and contributors (2016). TorchVision: PyTorch's Computer Vision library. GitHub. Available at: https://github.com/pytorch/vision [Accessed: March 2026].